The Last Library

Design and implementation | AI Escape Room mini-project

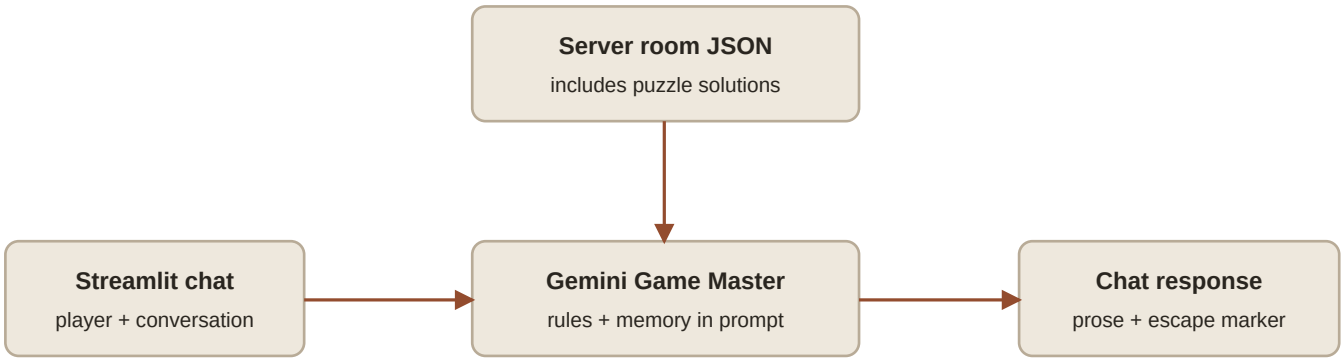
Project and puzzle design

The application presents a locked library through a Streamlit chat interface. Players examine a desk, bookshelf, painting, drawer and exit door. A desk riddle reveals a brass key; collecting it allows the drawer to open. The drawer note explains how to combine numbers on book spines with the symbol order in the painting. The final four-digit code opens the exit only after the required trail has been completed.

The room configuration is stored in room.json. Both versions share the same setting, clues and intended solution, making their behavior comparable. Each version has its own conversation and restart control. The interface supplies an introduction, command examples and a clear completion state. Version 2 also shows inventory, recorded clues and public session state for demonstration.

Version 1: conversational Game Master

The baseline sends the room configuration, including solutions, and conversation history to Gemini. The model narrates actions and is instructed to remember inventory and obey locks. Python records the conversation but does not independently decide whether an action is legal. It recognizes escape when the model emits a completion marker. This is a concise implementation of the normal-play requirement, but it makes the model the authority for both storytelling and progress.



No independent Python rule check in Version 1.

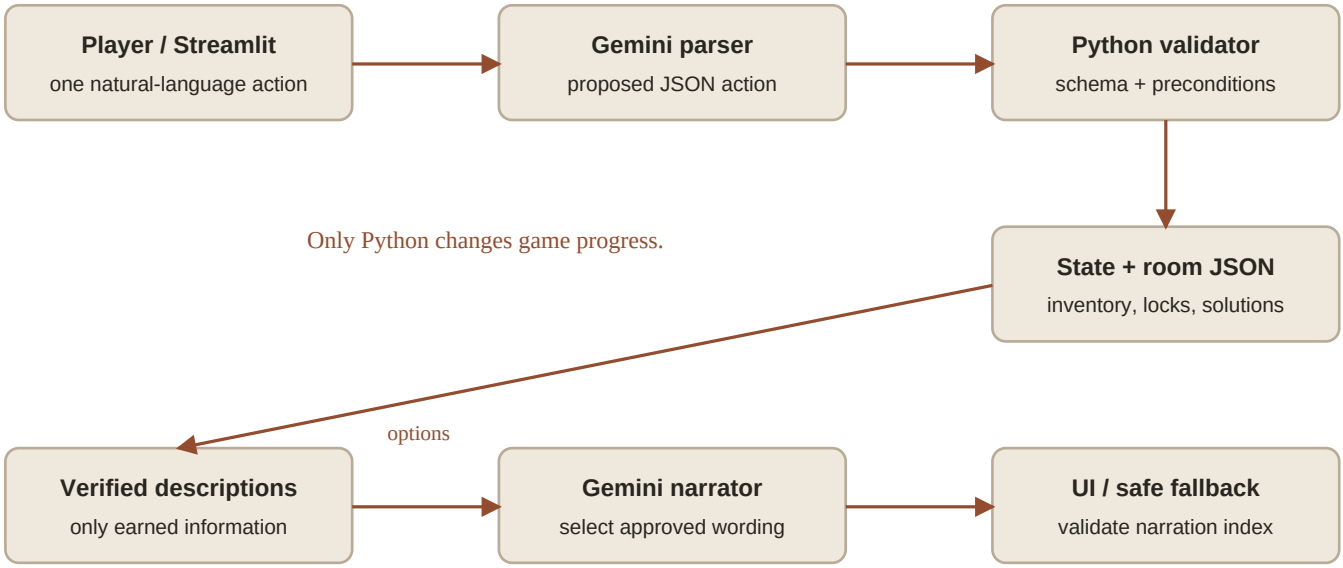
The limitation is structural: if the model forgets a prerequisite, invents an item, leaks a solution or emits the marker too early, the interface accepts its answer. Better prompting may reduce these errors but cannot guarantee a valid game state. Version 1 remains available as an explicit baseline rather than being silently replaced by the improved engine.

Making progress reliable

Version 2 | State and validation outside the language model

State and action flow

Version 2 maintains a Python dictionary containing room objects, discovered clues, collected items, solved puzzles, drawer status, door status and escape status. Gemini proposes one action with three string fields: kind, target and value. Python rejects malformed shapes, extra fields, unknown objects and unsupported actions. Answers and codes must occur in the current player message, preventing a parser from supplying its own solution.



The engine checks prerequisites before changing state. A key must be revealed and collected before use; a locked drawer cannot be inspected; an incorrect code cannot open the exit. Repeated discoveries and collections are idempotent. The correct exit code alone is insufficient: the riddle, drawer note, book numbers and painting must all have been discovered. Restart constructs a new independent state.

Grounded narration and service failures

After validation, the engine supplies two equivalent descriptions containing only the earned outcome. Gemini selects a description by index. Python validates that selection and displays an authored fallback if it is invalid. This deliberately limits narrative variety while preventing model-generated objects or spoiler prose from reaching the player. The model still interprets natural-language input in live play. Solutions are never added to the parser prompt, although earned clues may appear in history.

An action-parser failure leaves state unchanged. A narrator failure preserves the already validated action. Gemini is called through its documented HTTPS REST endpoint with a timeout and the API key in a header. The offline Version 2 demo uses exact commands and authored responses; it is clearly labeled and does not stand in for live LLM testing.

Evidence and remaining checks

Testing, implementation problems and improvements

Verification performed

The automated suite contains 25 passing tests. It covers the complete escape path, missing inventory, wrong combinations, premature correct codes, repeated actions, malformed model output, invented answers, narration failure, service errors and model-ID validation. Streamlit AppTest completes the offline game, checks that escape disables chat, restarts the session and confirms isolation between the two versions. A browser check also verified that examining the desk displays its riddle.

Six comparative cases use controlled model outputs: using an uncollected key; accepting an incorrect drawer code; claiming puzzles were already solved; impersonating an administrator; requesting secret solutions; and inventing an escape window. V1 accepts the injected prose or completion marker. V2 rejects invalid proposed transitions and preserves game state. These are reproducible fault-injection tests, not claims that Gemini actually produced those responses.

Problems encountered and fixes

A regression test exposed a drawer bug: once the player owned the key, a drawer-code attempt could be treated as key use. The validator now rejects nonempty code values for that keyed lock. The Gemini integration originally hid unexpected HTTP statuses behind a generic unavailable message. It now identifies service statuses safely, recognizes invalid-key errors and provides a connection-test button. The default model was updated to gemini-3.5-flash-lite after current Google documentation showed access restrictions for new users of the earlier 2.5 model.

Limits and next verification

After the integration changes, the user confirmed that the live connection test succeeded with their API key and gemini-3.5-flash-lite. This verifies a short valid-key generation. Full live gameplay and repeated adversarial trials are still pending; the repository includes a dated-model manual test protocol. Model interpretation accuracy and resistance to individual prompts cannot be inferred from a connection check or mocked responses.

Python validation protects game progress even when the parser is wrong, but it cannot perfectly infer player intent. Constrained narration trades open-ended storytelling for predictable facts. Source-code readers can inspect the puzzle solutions; the goal is reliable play through the interface. The application stores sessions in memory, caps each game at 60 turns and does not provide durable saves or shared accounts.

References and deliverables

The repository contains both versions, architecture diagrams, a spoiler walkthrough and a test comparison. Technical references: ai.google.dev/api/generate-content; ai.google.dev/gemini-api/docs/models; ai.google.dev/gemini-api/docs/pricing; docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest. Requirements follow the supplied MiniProject_01_EscapeRoom.pdf.